

```
cmake --build build/  
./build/Debug/task07.exe
```

Aufgabe07

- <https://docs.gl/>
 - <https://developer.nvidia.com/gpugems/gpugems/part-ii-lighting-and-shadows/chapter-11-shadow-map-antialiasing>
-

7.0) Erweitern Sie Ihr Programm aus Task06 um 4x4 PCF.

Damit Sie den Schattentest innerhalb der `texture()`-GLSL-Funktion durchführen können. Außerdem müssen Sie anstatt eines `sampler2D` einen `sampler2DShadow` im Fragment-Shadercode für Ihre `depthmap`-Textur verwenden, damit Sie einen `vec3` als Texturkoordinaten an `texture()` übergeben können.

Implementierung

- Im Fragment-Shader statt den `sampler2D` nimmt man einen `sampler2DShadow` und übergibt an die `texture()`-Funktion einen `vec3` mit den Texturkoordinaten (`s`, `t`, `r`), wobei `r` der Vergleichswert ist. Der Rückgabewert ist 1.0, wenn der Vergleichswert kleiner oder gleich dem Tiefenwert in der Shadowmap ist (Fragment ist beleuchtet). Ist der Vergleichswert größer als der Tiefenwert in der Shadowmap, ist der Rückgabewert 0.0 (Fragment ist im Schatten).
- `vec3 projCoords`

```
projCoords.z -= u_BiasAmount; // Bias hinzufügen  
shadow = 1 - texture(u_ShadowMap, projCoords.xyz);  
  
outColor = texColor * (1 - shadow);
```

7.1) GUI - PCF

Fügen Sie in der GUI eine Checkbox hinzu, mit der Sie PCF an- bzw. ausschalten können.

```
ImGui::Checkbox("Aktiviere PCF", &usePCF);
```

- Innerhalb vom Fragment-Shader übergibt man ein uniform bool `u_UsePCF`, um zu bestimmen, ob PCF aktiviert ist oder nicht. Je nachdem wird der Schattentest mit oder ohne PCF durchgeführt.
 - In der main wird dann entweder PCF durchgeführt oder der normale einmalige Schattentest
-

7.2) GUI - FPS

Fügen Sie in ihr GUI-Fenster eine Textanzeige hinzu, die Ihnen die Frametime und FPS darstellt. Dazu können Sie sich existierender ImGui Funktionalität bedienen.

Testen Sie nun folgendes Szenario bei ausgeschaltetem VSync: Starten Sie Ihr Programm mit unterschiedlichen Shadowmap-Größen und schalten Sie PCF ein/aus und notieren Sie die FPS für:

- 1024x1024
- 2048x2048
- 4096x4096
- 8192x8192

Shadowmap Texturgrößen.

Implementierung

- In der main loop wird die Frametime der letzten 60 Sekunden gemittelt und daraus die FPS berechnet, welche dann in der GUI angezeigt wird. Die Mittelung über 60 Sekunden verhindert die starke Schwankung der FPS-Anzeige, was ein sauberes Ablesen verhindert. Die FPS-Anzeige wird in der GUI mit `ImGui::Text()` dargestellt.

Performance

Shadow Map Größe	FPS ohne PCF	FPS mit PCF
1024×1024	1850	1815
2048×2048	1880	1855
4096×4096	1850	1850
8192×8192	1950	1900

- Getestet wurde auf einer RTX2080 (8GB VRAM) mit der Standardfenstergröße von 800x600, mit Blick auf einem Schatten. In den Nvidia-Einstellungen wurden die globalen 3D-Einstellungen für die Applikation deaktiviert, sodass V-Sync sowie eine FPS-Einschränkung von bspw. 165 FPS deaktiviert waren.
 - Der nicht signifikante Unterschied der FPS-Werte zwischen den Auflösungen und PCF an/aus ist wahrscheinlich darauf zurückzuführen, dass die Berechnungen noch nicht ins Gewicht gefallen sind, sowie durch mögliche versteckte Optimierungen von Nvidia, die die wirkliche Leistung der Grafikkarte nicht vollständig ausnutzen. Der Unterschied zwischen den Werten ist wahrscheinlich auf verschiedene Systemlasten zurückzuführen, die während der Tests aufgetreten sind.
-

7.3) Dokumentation

Logik von Percentage Closer Filtering (PCF):

- Für jedes Fragment innerhalb der Region wird der Schattentest durchgeführt und das Ergebnis (1.0 = Beleuchtet, 0.0 = im Schatten) auf die anderen Schattentests aufsummiert und der Durchschnitt gebildet. Das Ergebnis besagt "zu wie viel Prozent sich das untersuchte Fragment im Schatten befindet."

Frage 1.:

Macht es Sinn, Schatten vollständig in Schwarz zu rendern? Wenn ja, warum? Wenn nicht, warum nicht?

-> **Lösung:** Nein, wenn die verdeckenden Objekte nicht vollständig lichtundurchlässig sind, verfärben diese den Schatten.

Frage 2.:

Wie hat sich die Texturgröße auf die Performance Ihres Programms ausgewirkt? Hat PCF die Performance erhöht oder erniedrigt? Wenn ja, um wie viel Prozent?

-> **Lösung:** Die Texturgröße hat die Performance nicht wirklich beeinflusst. Interessanterweise stieg sie sogar bei 8192x8192 an. PCF hat die Performance leicht erniedrigt, aber nicht signifikant (ca. 30-50 FPS).

Frage 3.:

PCF ermöglicht zwar einen weicheren Übergang am Rand des Schattens, lässt aber (mind.) eine physikalische Begebenheit außer Acht. Welche ist das?

-> **Lösung:** PCF berücksichtigt nicht die Lichtstreuung. In der Realität wird das Licht durch die Atmosphäre und andere Partikel gestreut, was zu weicheren Schatten führt. Dieser Effekt wird durch PCF nicht berücksichtigt, da es nur die Tiefenwerte der Schattenkarte vergleicht und nicht die Lichtstreuung.

In Bezug auf die vorherige Übung:

Renderingartefakt 3.: Shadow/ Light Bleeding

- Effekt, bei dem fälschlicherweise Licht in Bereichen auftritt, die eigentlich im Schatten liegen sollten.
- Tritt auf, wenn die Resthelligkeit des Halbschattens eines Objekts als Lichtfaktor eines anderen Objekts interpretiert wird, obwohl dieses Objekt eigentlich im Kernschatten liegen sollte.

Lösung: Percentage Closer Filtering (PCF)

- PCF kann genutzt werden, um den Fehler zu kaschieren.
-

Ergebnis



(a) Schatten ohne PCF



(b) Schatten mit PCF

Figure 1: Vergleich zwischen mit und ohne PCF